

# **INFORME TÉCNICO DE ANÁLISIS Y AUTOMATIZACIÓN QA**

**Evaluación Técnica: Karate Framework  
(API) y Playwright + Cucumber BDD  
(Frontend)**

**BACHILLER EN INGENIERIA DE SISTEMAS E INFORMATICA**

**PIMINCHUMO FLORES, JOSE LUIS**

Índice

1. Resumen Ejecutivo del Análisis ..... 3

2. Análisis de Ejecución: Pruebas Backend con Karate Framework..... 3

3. Análisis de Arquitectura Frontend: Playwright + Cucumber (POM) ..... 4

4. Propuesta de Mejora: Integración Continua (CI/CD) ..... 6

## 1. Resumen del Análisis

Para el desarrollo y resolución de este análisis técnico, utilicé Visual Studio Code como entorno integrado (IDE) principal para la configuración, compilación y ejecución satisfactoria de dos soluciones de pruebas automatizadas: por un lado, en la capa de servicios REST (Backend API), configuré Java 17, específicamente esta versión de Java ya que con versiones más actuales falla y Apache Maven para resolver las dependencias del archivo pom.xml y ejecutar la suite de Karate Framework mediante mvn clean test, validando el comportamiento y los contratos sobre el endpoint de PetStore a través de sus reportes HTML; y por otro lado, en la capa web (Frontend E2E), configuré el runtime de Node.js, gestioné las dependencias con npm install y descargué los binarios de navegación con npx playwright install, ejecutando la suite en la terminal con npm run test-dev para validar el flujo de interfaz bajo el patrón de diseño Page Object Model (POM) con Cucumber BDD y TypeScript.

Ambas pruebas fueron compiladas y ejecutadas de forma satisfactoria en un entorno local.

## 2. Análisis de Ejecución: Pruebas Backend con Karate Framework

Durante la ejecución de la suite mediante Apache Maven y JUnit 5, se procesó el escenario de prueba 'valida la creación de una mascota' sobre el endpoint público PetStore.

| Gherkin                                                                                                                 | Función Técnica      | Interpretación y Validación                                                                                                                                             |
|-------------------------------------------------------------------------------------------------------------------------|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>url urlBase<br/>Given path '/pet'</li></ul>                                       | Construcción de URL  | Consume la URL base configurada en karate-config.js y concatena el path relativo para apuntar a https://petstore.swagger.io/v2/pet.                                     |
| <ul style="list-style-type: none"><li>def json =<br/>read('...createPet.json')<br/>set json.name = 'Firulais'</li></ul> | Inyección Dinámica   | Lee el payload modelo en formato JSON y actualiza dinámicamente el valor del campo 'name' con el parámetro 'Firulais' antes del envío.                                  |
| <ul style="list-style-type: none"><li>When method post</li></ul>                                                        | Llamada HTTP POST    | Envía la petición POST con cabecera Content-Type: application/json. El servidor responde con estado 200 en 297 ms devolviendo el registro con ID asignado.              |
| <ul style="list-style-type: none"><li>Then status 200<br/>And match response.name == 'Firulais'</li></ul>               | Aserciones (Asserts) | Valida el contrato y el comportamiento: verifica que el código de estado HTTP sea 200 (éxito) y que la propiedad 'name' del JSON devuelto sea exactamente 'Firulais'.   |
| <ul style="list-style-type: none"><li>And print 'Mascota creada: ',<br/>response.name</li></ul>                         | Trazabilidad         | Imprime en consola y en el reporte HTML de Karate el nombre recibido ("Mascota creada: Vaguito" y "Mascota creada: Firulais") para control y auditoría de la ejecución. |

```
Karate version: 1.0.0
=====
elapsed: 2.96 | threads: 1 | thread time: 1.39
features: 1 | skipped: 0 | efficiency: 0.47
scenarios: 2 | passed: 2 | failed: 0
=====

HTML report: (paste into browser to view) | Karate version: 1.0.0
file:///C:/Users/USUARIO/Desktop/CV%20Jose/NTTDATA/QA%20Test/karateNttdataDemo/target/karate-reports/karate-summary.html
```



2

0

Scenarios

2026-10-05 02:13:21 p. m.

[1.1:19] valida la creación de una mascota

[1.2:20] valida la creación de una mascota

Summary | Tags | Feature: [resources/features/createPetStore.feature](#) | Ejemplo de karate

|                                                                   |  |          |
|-------------------------------------------------------------------|--|----------|
| <b>createPet</b>                                                  |  | ms: 1086 |
| Scenario: [1.1:19] valida la creación de una mascota              |  |          |
| >> Background:                                                    |  |          |
| 8 Given path '/pet'                                               |  | 4        |
| 9 * def json = read('classpath:resources/request/createPet.json') |  | 42       |
| 10 * set json.name = 'Vaguito'                                    |  | 1        |
| 11 And request json                                               |  | 0        |
| 12 When method post                                               |  | 1029     |
| 13 Then status 200                                                |  | 0        |
| 14 And match response.name == 'Vaguito'                           |  | 8        |
| 15 And print 'Mascota creada: ', response.name                    |  | 2        |

|                                                                   |  |         |
|-------------------------------------------------------------------|--|---------|
| <b>createPet</b>                                                  |  | ms: 303 |
| Scenario: [1.2:20] valida la creación de una mascota              |  |         |
| >> Background:                                                    |  |         |
| 4 * url baseUrl                                                   |  | 0       |
| 14:13:21.294 karate.env system property was: null                 |  |         |
| 8 Given path '/pet'                                               |  | 0       |
| 9 * def json = read('classpath:resources/request/createPet.json') |  | 1       |
| 10 * set json.name = 'Firulais'                                   |  | 1       |
| 11 And request json                                               |  | 0       |
| 12 When method post                                               |  | 299     |
| 13 Then status 200                                                |  | 0       |
| 14 And match response.name == 'Firulais'                          |  | 0       |
| 15 And print 'Mascota creada: ', response.name                    |  | 0       |
| 14:13:21.599 [print] Mascota creada: Firulais                     |  |         |

### 3. Análisis de Arquitectura Frontend: Playwright + Cucumber (POM)

La automatización de interfaz de usuario implementa el patrón Page Object Model (POM) con BDD en TypeScript:

- Especificación BDD (.feature): Casos de prueba descritos en lenguaje Gherkin (Given/When/Then) orientados a negocio.
- Clases Page Object (loginPage.ts, dashboardPage.ts): Centralizan los localizadores (selectores estables) y las acciones sobre la UI (click, fill) desacoplándolos de los tests.
- Step Definitions: Conectan los pasos de Gherkin con los métodos de las clases Page Object.
- Ciclo de Vida (hooks.ts): Inicializan el contexto del navegador antes de cada prueba y capturan evidencias/screenshots ante fallos.
- Ejecución Cross-Browser: Configurado en package.json para ejecución selectiva en Chromium, Firefox y WebKit.

```

PS C:\Users\USUARIO\Desktop\CV Jose\WTTDATA\QA Test\playwright-pacifico> npm run test-dev

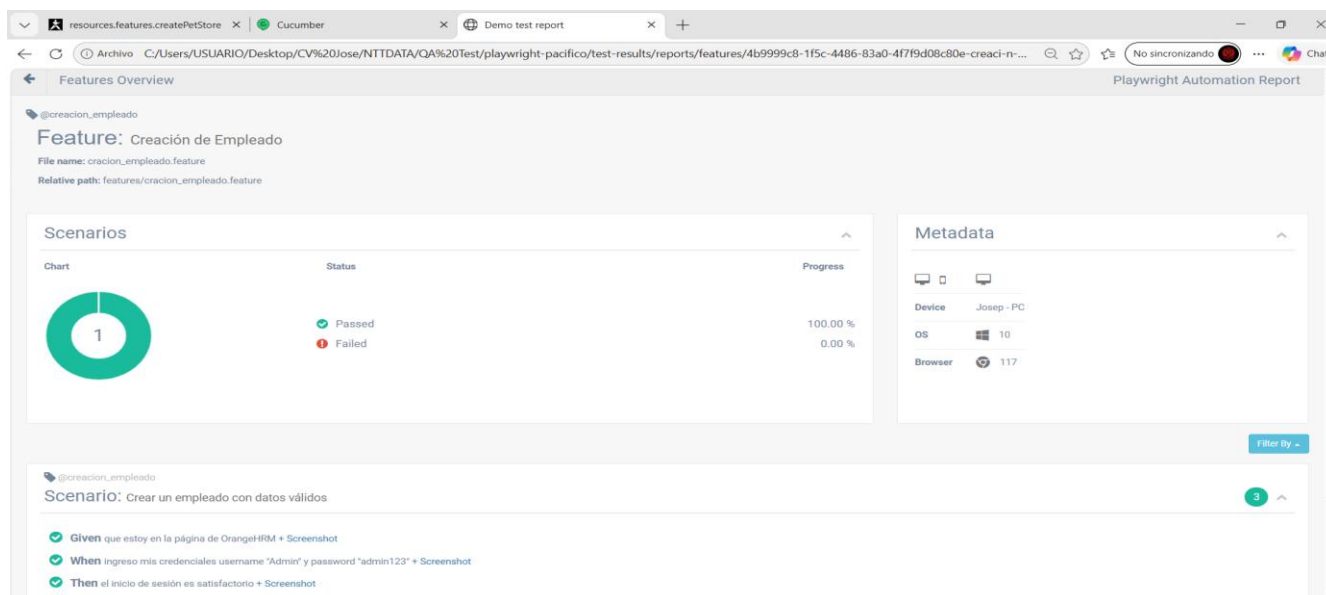
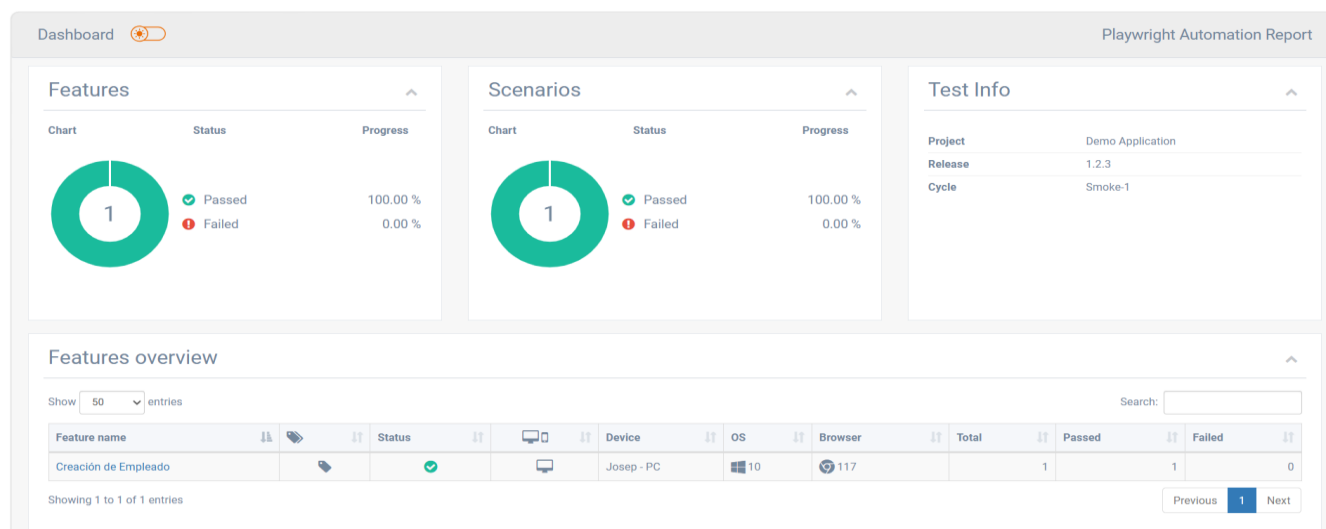
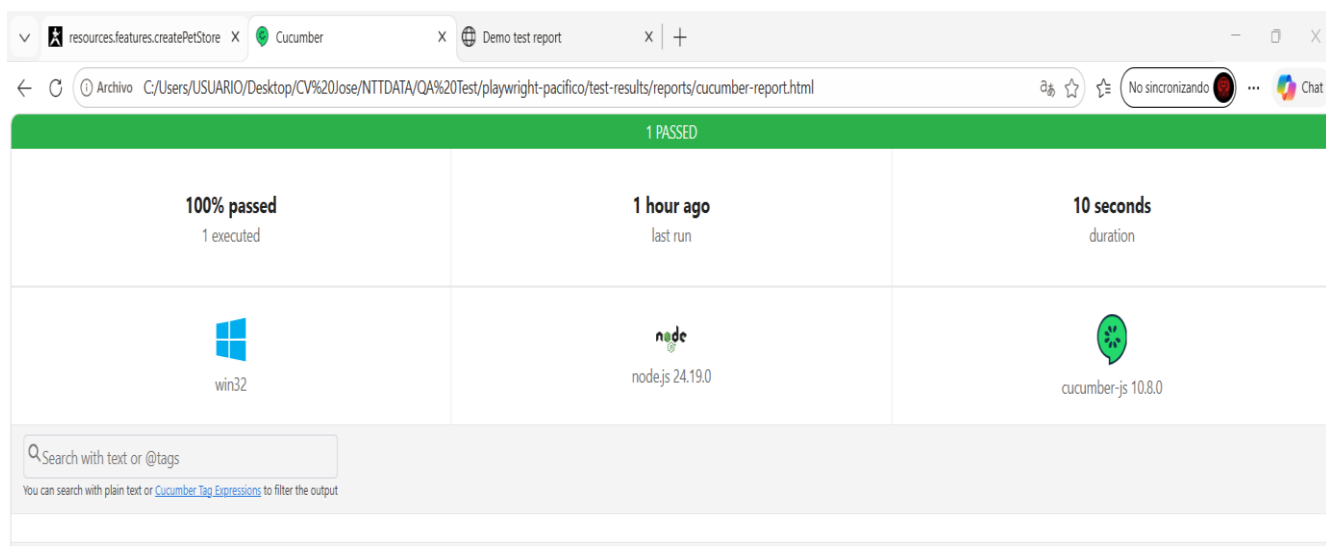
> playwright-pacifico@1.0.0 test-dev
> cross-env ENV=dev BROWSER=chrome cucumber-js --config=config/cucumber.js

This Node.js version (v24.19.0) has not been tested with this version of cucumber; it should work normally, but please raise an issue if you see anything unexpected.
1 scenario (1 passed)
3 steps (3 passed)
0m10.080s (executing steps: 0m06.997s)

> playwright-pacifico@1.0.0 posttest-dev
> npx ts-node src/helper/report.ts

=====
Multiple cucumber HTML report generated in:
C:\Users\USUARIO\Desktop\CV Jose\WTTDATA\QA Test\playwright-pacifico\test-results\reports\index.html
=====

```



#### 4. Propuesta de Mejora: Integración Continua (CI/CD)

Como valor agregado para el equipo de QA, propongo integrar ambas suites en un pipeline automatizado (GitHub Actions / Azure DevOps):

1. Backend (Fast Feedback): Ejecución del comando 'mvn clean test' en cada Pull Request para validar que los endpoints mantengan sus contratos sin errores.
2. Frontend (Regresión E2E): Ejecución programada de 'npm run test-dev' para validar flujos críticos sobre el navegador en despliegues a entornos de prueba.
3. Centralización de Reportes: Publicación automática de reportes HTML (Karate Reports y Multiple Cucumber HTML Reporter) como artefactos del pipeline.